CSA0604 – DESIGN AND ANALYSIS OF ALGORITHMS

LAB EXPERIMENTS -SESSION 2

NAME : Shree Ranjane K

REG NO: 192471052

1. Program to Find Unique Elements in an Array in C

```
#include <stdio.h>

int main() {
    int n, i, j, k = 0;
    int arr[100], unique[100], found;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for (i = 0; i < n; i++) {
        found = 0;

        for (j = 0; j < k; j++) {
            if (arr[i] == unique[j]) {
                found = 1;
                break;
            }
        }
```

```
        if (found == 0) {
            unique[k] = arr[i];
            k++;
        }
    }

    printf("Unique elements: ");
    for (i = 0; i < k; i++) {
        printf("%d ", unique[i]);
    }

    return 0;
}
```

Example Output

Enter number of elements: 6

Enter elements: 1 2 2 3 4 3

Unique elements: 1 2 3 4

Complexity Analysis

- Time Complexity: $O(n^2)$— nested loops compare elements.
- Auxiliary Space Complexity: $O(n)$— the unique array stores up to $n$elements.



2. Program to Sort an Array Using Bubble Sort in C

```
#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
```

```
    return 0;
}
```

Input

Enter number of elements: 5

Enter elements: 5 2 4 1 3

Output

Sorted array: 1 2 3 4 5

- Best case: $O(n^2)$— in this basic implementation, comparisons are performed even if the array is already sorted.
- Average case: $O(n^2)$
- Worst case: $O(n^2)$— when the array is in reverse order.

Space complexity: $O(1)$, because Bubble Sort uses only a temporary variable and sorts the array in place.



3.Program to Search an Element Using Binary Search in C

```
#include <stdio.h>

int main() {
    int arr[100], n, x, i, low, high, mid;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter sorted elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter search element: ");
    scanf("%d", &x);

    low = 0;
    high = n - 1;

    while (low <= high) {
        mid = (low + high) / 2;

        if (arr[mid] == x) {
            printf("Element found at position %d", mid + 1);
            return 0;
        } else if (arr[mid] < x) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    printf("Element not found");
    return 0;
}
```

Input

Enter number of elements: 5

Enter sorted elements: 10 20 30 40 50

Enter search element: 30

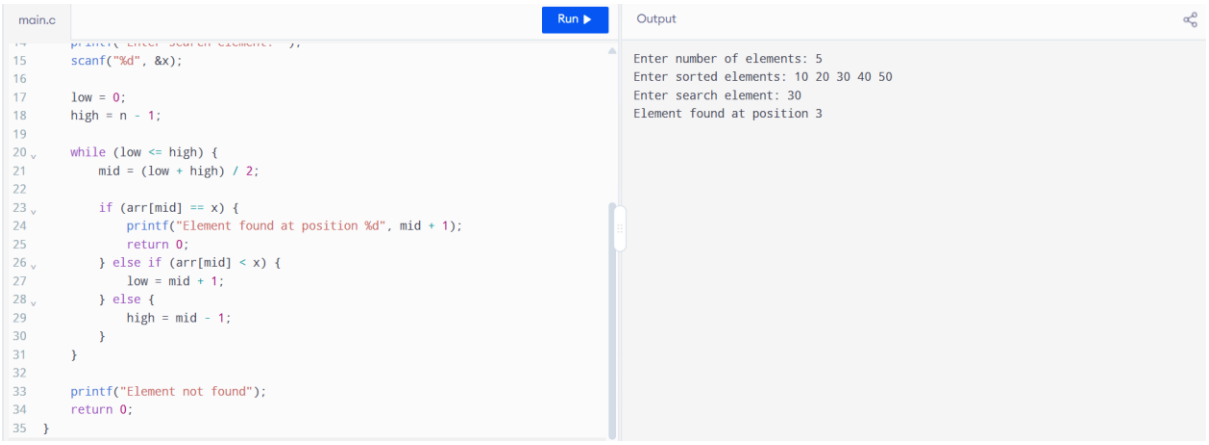
Output

Element found at position 3

Time Complexity Analysis

- Best case: $O(1)$— the element is found at the middle in the first comparison.
- Average case: $O(\log n)$
- Worst case: $O(\log n)$— the search space is halved in every iteration.

Space complexity: $O(1)$, because the iterative binary search uses a constant amount of extra space.



4. C Program to Sort an Array Using Merge Sort

```
#include <stdio.h>


void merge(int a[], int l, int m, int r) {
   int temp[1000], i = l, j = m + 1, k = 0;


   while (i <= m && j <= r) {
      if (a[i] <= a[j])
```

```
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= m)
        temp[k++] = a[i++];

    while (j <= r)
        temp[k++] = a[j++];

    for (i = l, k = 0; i <= r; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int n, a[1000], i;

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Input

Enter number of elements: 6

Enter elements: 5 2 8 1 3 7

Output

Sorted array: 1 2 3 5 7 8

Time and Space Complexity

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$
- Auxiliary space: $O(n)$for the temporary array, plus $O(\log n)$recursion stack space.

main.c

Run ▶

```c
}

int main() {
    int n, a[1000], i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Output

```
Enter number of elements: 6
Enter elements: 5 2 8 1 3 7
Sorted array: 1 2 3 5 7 8
```